# Lo que la red espera y lo que obtiene: análisis reproducible de los valores Q de una política DQN para Pong con y sin acciones pegajosas




Leonar Socarrás Molina

John Jairo Serrano Cifuentes

Brezhnev Joya Miranda

Gabriel Alonso Lizano Alvarado

Bryan Johann Aranzazu Medina

Maestría en Inteligencia Artificial, Facultad de Ingeniería, Universidad de La Sabana

EFMIA-SAPR20265 Simulación y Aprendizaje por Refuerzo

Unidad 3, Taller 2 - Resolver un ambiente nuevo de Gymnasium con DQN

Profesor Emilio Muñoz Pérez

Octubre de 2026

Grupo 12

**Lo que la red espera y lo que obtiene: análisis reproducible de los valores Q de una política DQN para Pong con y sin acciones pegajosas**

El repositorio grupal del Taller 2 entrena un agente Deep Q-Network que aprende a jugar Pong a partir de la pantalla, con el preprocesamiento y la red convolucional de Mnih et al. (2015). Su reflexión de resultados (sección 6 del README) sostiene tres afirmaciones sobre el comportamiento interno de los dos mejores modelos: que anotan con un ritmo fijo de 77 o 78 pasos; que la red asigna valores Q casi iguales a las acciones que en Pong tienen el mismo efecto; y que sobreestima sus propios valores entre un 17 % y un 20 %, el sesgo que van Hasselt et al. (2016) atribuyen al operador máximo del objetivo de Bellman y que Thrun y Schwartz (1993) ya habían descrito para la aproximación de funciones. A diferencia de las métricas de recompensa, ninguna de esas tres mediciones tenía un script en el repositorio, de modo que no era posible verificarlas ni repetirlas.

Por otra parte, el aporte de robustez del grupo mostró que, al activar las acciones pegajosas que Machado et al. (2018) propusieron como estándar de evaluación, la ventaja de los modelos cae de +21 a +10,5 y +7,4. Esa curva describe el efecto en el marcador, pero no lo que ocurre dentro de la red: si sus valores Q anticipan el peor desempeño o si siguen describiendo el emulador determinista en el que se entrenaron. Este aporte individual (a) convierte las tres mediciones de la sección 6 en un script reproducible, *scripts/analisis_q.py*; (b) verifica con él las cifras publicadas por el grupo; y (c) repite las mediciones con acciones pegajosas de 0,25 para comparar lo que la red espera con lo que realmente obtiene. Los cambios se propusieron mediante un pull request desde la rama *aporte-bryan-analisis-q* del fork BryanAranzazu/pong_dqn hacia el repositorio glizano/pong_dqn.

# Método

## Modelos evaluados y entorno

Se analizaron los mejores modelos de las dos corridas del grupo, entrenadas durante un millón de pasos con el emulador determinista: la semilla de entrenamiento 1 (checkpoint del paso 800 000, Colab) y la semilla 0 (checkpoint del paso 900 000, Mac). El entorno es `ALE/Pong-v5` de Gymnasium (Towers et al., 2024) con el preprocesamiento del repositorio: salto de cuatro cuadros con máximo de los dos últimos, escala de grises, 84 × 84 píxeles, hasta 30 *no-ops* (acciones nulas) al reiniciar un episodio y apilado de cuatro cuadros. Cada modelo jugó cinco partidos con política voraz ($\varepsilon = 0$) en dos condiciones: sin acciones pegajosas (probabilidad de repetición 0, aunque el valor por defecto de `ALE/Pong-v5` es 0,25) y con probabilidad de repetición de 0,25. Se usaron las semillas de evaluación 10 000 a 10 004, las cinco primeras de la evaluación final del grupo. Esas mismas semillas se usaron para elegir el checkpoint durante el entrenamiento, por lo que las cifras no son independientes de la selección. Las cuatro condiciones suman 20 partidos y 44 307 pasos de decisión.

## Mediciones

En cada paso $t$ se registraron los seis valores $Q(s_t, \cdot)$ de la red en línea, la acción elegida $a_t = \text{argmax}_a\, Q(s_t, a)$ y la recompensa $r_t \in \{-1, 0, +1\}$. Con esos registros se calcularon tres grupos de medidas, que corresponden a las tres afirmaciones de la sección 6 del README.

*Ritmo de anotación.* Se contó el número de pasos entre dos puntos a favor consecutivos dentro de un mismo partido. Si la política repite siempre la misma jugada, ese intervalo debe ser constante. El intervalo incluye los puntos del rival que ocurran entre ambos, por lo que un punto perdido en medio lo alarga.

*Acciones redundantes.* En Pong, FIRE produce el mismo efecto que NOOP, RIGHTFIRE el mismo que RIGHT y LEFTFIRE el mismo que LEFT (sección 2.4 del README). Se calculó la diferencia absoluta media $|Q(s, a) - Q(s, a')|$ sobre los tres pares de acciones con el mismo efecto y sobre los doce pares con efecto distinto, promediada en todos los pasos. También se contó cuántas veces se eligió cada acción.

*Sobreestimación.* Para cada paso se calculó el retorno descontado que el agente obtuvo realmente a partir de ese estado, $G_t = \Sigma_{k \geq 0}\, \gamma^k\, r_{t+k}$, con $\gamma = 0{,}99$, el mismo factor del entrenamiento (Sutton & Barto, 2018). Como el partido termina de verdad al llegar a 21 puntos, la suma no necesita completarse con una estimación al final. La diferencia entre la media de $Q(s_t, a_t)$ y la media de $G_t$ mide cuánto se aleja la estimación de la red de lo que consigue su propia política. Con el emulador determinista y la política voraz, el retorno observado desde cada estado es una consecuencia determinista de la política, de manera que esa diferencia refleja sesgo de la estimación y no ruido del entorno.

**Cambios en el código**

El script *scripts/analisis_q.py* reutiliza las funciones del paquete (*crear_entorno* y *AgenteDQN.cargar*) y las semillas de *pong_dqn.evaluar*, por lo que no modifica el código de entrenamiento ni el de evaluación. Recibe el modelo, la etiqueta de la corrida, el número de partidos y el nivel de acciones pegajosas, y guarda los resultados en *resultados/analisis_q/<etiqueta>.json*. El aporte incluye los cuatro archivos de resultados, un párrafo nuevo en la sección 6 del README, los comandos de reproducción en la sección 9, la entrada del script en la sección 10 y el registro en el CHANGELOG. Antes de proponer el cambio se verificó que ruff no reporta errores y que las 10 pruebas del repositorio pasan. Cada condición corre en CPU en pocos minutos.

## Resultados

Sin acciones pegajosas, el script reproduce las cifras de la sección 6 del README (Tabla 1). Los dos modelos ganan los cinco partidos 21 a 0, y todos los intervalos entre puntos a favor miden 77 o 78 pasos (100 de 100 en cada modelo). El *Q* medio de la acción elegida es 1,48 y 1,42, frente a un retorno observado de 1,23 y 1,22: una sobreestimación de 0,25 (20 %) y 0,20 (17 %), la misma que informa el grupo. La diferencia media de *Q* entre acciones con el mismo efecto, 0,015 y 0,016, es unas tres veces menor que entre acciones con efecto distinto, 0,043 y 0,050. Estos dos últimos valores quedan algo por debajo del rango de 0,046 a 0,059 citado en el README, que se midió en un solo partido y posiblemente con otra agrupación de pares; la conclusión no cambia.

**Tabla 1**

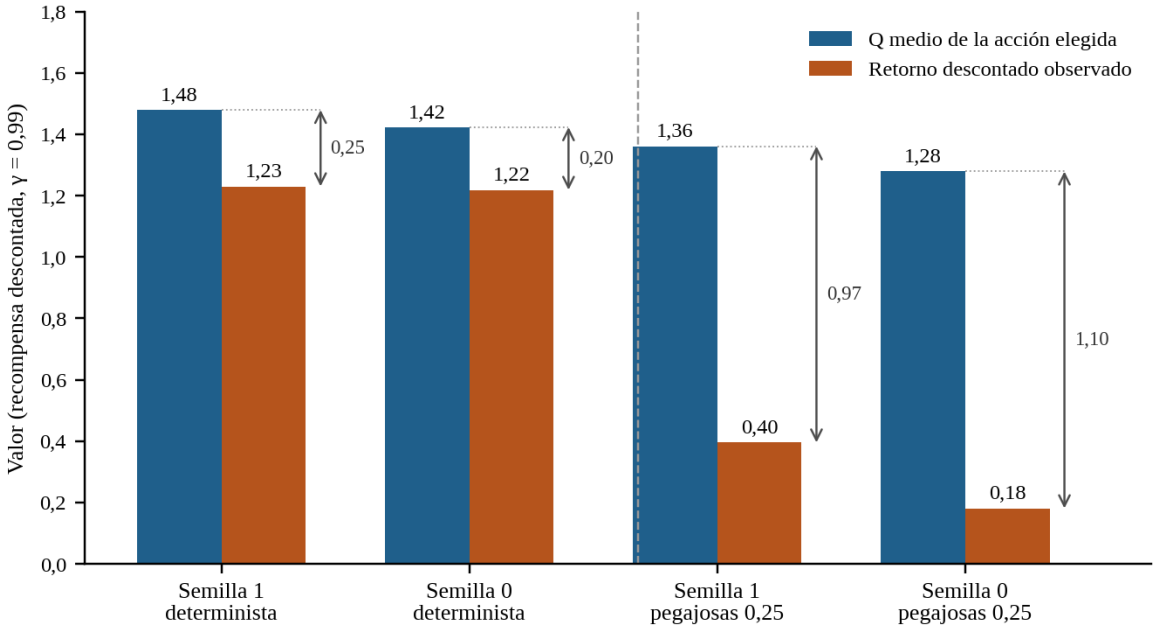
*Mediciones sobre los valores Q por modelo y condición (5 partidos por condición)*

| Medición | Semilla 1, determinista | Semilla 0, determinista | Semilla 1, pegajosas 0,25 | Semilla 0, pegajosas 0,25 |
|---|---|---|---|---|
| Recompensa media por partido | +21,00 | +21,00 | +10,40 | +5,60 |
| Pasos entre puntos a favor, media [mín.; máx.] | 77,87 [77; 78] | 77,79 [77; 78] | 123,19 [77; 380] | 142,35 [77; 505] |
| Intervalos de 77 o 78 pasos | 100/100 | 100/100 | 59/100 | 43/98 |
| \|ΔQ\| acciones con el mismo efecto | 0,015 | 0,016 | 0,025 | 0,030 |
| \|ΔQ\| acciones con efecto distinto | 0,043 | 0,050 | 0,055 | 0,056 |
| Q medio de la acción elegida | 1,480 | 1,423 | 1,361 | 1,279 |
| Retorno descontado observado | 1,230 | 1,219 | 0,396 | 0,181 |
| Brecha Q − G | 0,249 | 0,204 | 0,965 | 1,098 |

*Nota.* Semilla 1: modelo del paso 800 000 (Colab). Semilla 0: modelo del paso 900 000 (Mac). Semillas de evaluación 10 000 a 10 004, $\varepsilon = 0$. |ΔQ| = diferencia absoluta media entre los valores Q de dos acciones, promediada en todos los pasos; 3 pares con el mismo efecto y 12 con efecto distinto. Retorno descontado con $\gamma = 0{,}99$. Con acciones pegajosas, la semilla 0 suma 98 intervalos entre sus cinco partidos. El JSON guarda solo los intervalos más frecuentes, por lo que el total y el conteo de intervalos de 77 o 78 pasos deben calcularse y guardarse en analisis_q.py para poder verificarlos. Fuente: resultados/analisis_q/*.json.

**Figura 1**

*Valor esperado por la red frente al retorno observado, con y sin acciones pegajosas*




*Nota.* Barras azules: media de Q(s, a) de la acción elegida. Barras naranjas: media del retorno descontado que el agente obtuvo desde esos mismos estados. Las flechas indican la brecha entre ambas. Cinco partidos por condición.

Con acciones pegajosas de 0,25 el panorama cambia (Figura 1). El *Q* medio de la acción elegida baja poco, a 1,36 y 1,28, es decir, un 8 % y un 10 % menos que con el emulador determinista. El retorno observado, en cambio, cae a 0,40 y 0,18, una reducción del 68 % y del 85 %. La brecha entre lo que la red espera y lo que obtiene pasa de 0,25 y 0,20 a 0,97 y 1,10: se multiplica por casi cuatro en la semilla 1 y por más de cinco en la semilla 0. La recompensa media de estos cinco partidos, +10,40 y +5,60, es coherente con la de la evaluación de 30 partidos del grupo (+10,50 y +7,40).

El ritmo de anotación no desaparece con el ruido. En la semilla 1, 59 de los 100 intervalos siguen midiendo exactamente 77 o 78 pasos, y en la semilla 0, 43 de los 98. El resto se dispersa hasta 380 y 505 pasos, con un segundo grupo de intervalos entre 140 y 165 pasos (el 17 % y el 20 %), aproximadamente el doble del ritmo base. La separación entre acciones también se

vuelve menos nítida: la diferencia de $Q$ entre acciones con el mismo efecto sube a 0,025 y 0,030, mientras que entre acciones con efecto distinto apenas cambia (0,055 y 0,056). La razón entre ambas pasa de 0,35 y 0,33 a 0,46 y 0,53.

La Tabla 2 muestra cómo usa cada modelo las seis acciones. Las dos redes reparten de forma opuesta los pares equivalentes: la semilla 1 elige FIRE en el 20,3 % de los pasos y NOOP solo en el 5,7 %, mientras que la semilla 0 elige NOOP en el 25,4 % y FIRE en el 8,0 %. Sumando cada par, ambas usan los tres efectos en proporciones comparables: quedarse quieto entre el 26 % y el 33 % de los pasos, subir entre el 30 % y el 31 %, y bajar entre el 37 % y el 43 %.

**Tabla 2**

*Frecuencia de uso de cada acción (% de los pasos de decisión)*

| Acción | Semilla 1, determinista | Semilla 0, determinista | Semilla 1, pegajosas 0,25 | Semilla 0, pegajosas 0,25 |
|---|---|---|---|---|
| 0 NOOP | 5,7 | 25,4 | 8,0 | 20,0 |
| 1 FIRE (= NOOP) | 20,3 | 8,0 | 15,4 | 10,9 |
| 2 RIGHT (subir) | 20,3 | 16,5 | 20,9 | 16,5 |
| 3 LEFT (bajar) | 18,2 | 15,7 | 16,4 | 14,1 |
| 4 RIGHTFIRE (= RIGHT) | 10,4 | 13,3 | 20,4 | 17,6 |
| 5 LEFTFIRE (= LEFT) | 25,0 | 21,2 | 18,9 | 20,9 |

*Nota.* Porcentaje sobre 8 222, 8 385, 12 925 y 14 775 pasos de decisión, respectivamente. Las columnas pueden no sumar 100 por redondeo.

## Discusión

El primer resultado es de reproducibilidad. Las tres afirmaciones de la sección 6 del README eran las únicas del informe sobre el comportamiento interno de la red y las únicas sin un procedimiento que permitiera repetirlas. Ahora se generan con un comando, y sus valores coinciden con los publicados salvo por una diferencia menor en la escala de $|\Delta Q|$ que no altera la

conclusión. Las afirmaciones de este tipo suelen ser las más interesantes de una reflexión y, a la vez, las más fáciles de dejar sin verificar.

La sobreestimación de 0,20 a 0,25 en el emulador determinista es compatible con el sesgo optimista del operador máximo (Thrun & Schwartz, 1993; van Hasselt et al., 2016): el objetivo de Bellman toma el mayor de seis valores estimados con error, y ese máximo tiende a ser mayor que el valor verdadero. Como en esta condición el retorno observado no tiene ruido del entorno, la brecha es sesgo de la estimación. No afecta al juego porque la política solo depende del orden entre acciones, no del valor absoluto, pero confirma que los valores Q de DQN no deben leerse como predicciones calibradas.

La brecha con acciones pegajosas es un fenómeno distinto. La red se entrenó en un proceso de decisión donde la acción elegida siempre se ejecuta. Con acciones pegajosas cambia la dinámica del entorno, porque una de cada cuatro veces se repite la acción anterior, pero nada en la observación lo indica y la red sigue estimando el valor de un mundo en el que su acción se cumple. $Q(s, a)$ aproxima el retorno si se ejecuta $a$ y luego se sigue la política voraz sin interferencias; esa premisa falla en una cuarta parte de los pasos. Por eso la brecha se multiplica por cuatro o cinco mientras el valor esperado casi no se mueve: la red no anticipa que está jugando peor. En términos prácticos, sus valores Q no sirven como indicador de confianza fuera de las condiciones de entrenamiento. Este resultado complementa desde dentro de la red la crítica de Machado et al. (2018): evaluar sin estocasticidad no solo infla el marcador, también oculta que la representación aprendida depende del determinismo.

La persistencia de los intervalos de 77 o 78 pasos indica que, aun con ruido, la política sigue intentando la misma jugada, y que esta funciona siempre que ninguna acción decisiva quede anulada. Cuando falla, el intercambio continúa y el punto suele llegar más tarde, lo que es

compatible con el grupo de intervalos cercano al doble del ritmo base. Esta lectura ayuda a entender por qué la curva de robustez del grupo es gradual: cada punto se resuelve como un intento de la jugada memorizada que puede salir bien o mal, y la probabilidad de que salga mal aumenta con el nivel de acciones pegajosas. Es una interpretación plausible, no demostrada, porque las mediciones no identifican en qué paso se interfirió la jugada.

El uso opuesto de las acciones equivalentes en las dos semillas confirma que la red aprendió la equivalencia funcional, pero que la elección dentro de cada par depende de diferencias de $Q$ del orden de 0,015, determinadas por la inicialización y la historia de entrenamiento. Dos redes con el mismo desempeño llegaron a "nombres" distintos para la misma acción. El aumento de $|\Delta Q|$ entre acciones equivalentes con ruido sugiere que la red es menos coherente en los estados que visita con acciones pegajosas, que no abundaban en su experiencia de entrenamiento.

El análisis tiene límites claros. Usa cinco partidos por condición, frente a los 30 de la evaluación final del grupo, y no reporta intervalos de confianza; las cifras con acciones pegajosas deben leerse como estimaciones puntuales. Hay un solo modelo por semilla, así que las diferencias entre semillas no pueden atribuirse a la semilla, al equipo ni al punto de control. Con acciones pegajosas, además, el $Q$ registrado corresponde a la acción elegida, que no siempre es la ejecutada, y las mediciones no separan cuánta de la brecha se debe a esa discrepancia y cuánta a los estados nuevos que el ruido produce.

## Conclusiones y trabajo futuro

Las tres afirmaciones de la reflexión del grupo sobre los valores Q quedan verificadas y son reproducibles con un comando. En el emulador determinista la red sobreestima su retorno en 0,20 a 0,25, un sesgo esperable que no afecta la política. Con acciones pegajosas de 0,25, el valor

esperado casi no cambia mientras el retorno real cae entre un 68 % y un 85 %, y la brecha se multiplica por cuatro o cinco: la red conserva los valores de un entorno determinista que ya no existe y no anticipa su peor desempeño.

El script deja preparadas las mediciones para los dos experimentos que propone el aporte de robustez del grupo. Si una red entrenada con *--sticky 0.25* aprende la dinámica estocástica, su brecha evaluada con acciones pegajosas debería acercarse a la del emulador determinista; basta con ejecutar *analisis_q.py --sticky 0.25* sobre el nuevo modelo. Si una corrida con Double DQN reduce el sesgo del máximo como predicen van Hasselt et al. (2016), su brecha determinista debería quedar por debajo de 0,20. Ambos resultados serían comparables directamente con la Tabla 1.

## Referencias

Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., & Bowling, M. (2018). Revisiting the Arcade Learning Environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research, 61*, 523–562. https://doi.org/10.1613/jair.5699

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature, 518*(7540), 529–533. https://doi.org/10.1038/nature14236

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Thrun, S., & Schwartz, A. (1993). Issues in using function approximation for reinforcement learning. En M. Mozer, P. Smolensky, D. Touretzky, J. Elman, & A. Weigend (Eds.), *Proceedings of the 1993 Connectionist Models Summer School* (pp. 255–263). Lawrence Erlbaum.

Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J. J., Tan, H., & Younis, O. G. (2024). *Gymnasium: A standard interface for reinforcement learning environments* (arXiv:2407.17032). arXiv. https://doi.org/10.48550/arXiv.2407.17032

van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence, 30*(1), 2094–2100. https://doi.org/10.1609/aaai.v30i1.10295